Text Injection for Capitalization and Turn-Taking Prediction in Speech Models

Shaan Bijwadia[1], *Shuo-yiin Chang*[1], *Weiran Wang*[1], *Zhong Meng*[1], *Hao Zhang*[1], *Tara N. Sainath*[1]

[1]Google, USA

{shaanb, shuoyiin, weiranwang, haozhang, zhongmeng, tsainath}@google.com

arXiv:2308.07395v1 [cs.CL] 14 Aug 2023

Abstract

Text injection for automatic speech recognition (ASR), wherein unpaired text-only data is used to supplement paired audio-text data, has shown promising improvements for word error rate. This study examines the use of text injection for auxiliary tasks, which are the non-ASR tasks often performed by an E2E model. In this work, we use joint end-to-end and internal language model training (JEIT) as our text injection algorithm to train an ASR model which performs two auxiliary tasks. The first is capitalization, which is a de-normalization task. The second is turn-taking prediction, which attempts to identify whether a user has completed their conversation turn in a digital assistant interaction. We show results demonstrating that our text injection method boosts capitalization performance for long-tail data, and improves turn-taking detection recall.

Index Terms: speech recognition, text injection, auxiliary tasks

1. Introduction

Automatic speech recognition (ASR) has long been an integral part of important technologies, including voice dictation, digital assistants, and video captioning [1]. While ASR systems are typically evaluated based on word error rate (WER), this is not the only metric of concern in production applications; several "auxiliary tasks" must be integrated with the ASR task in a full system. These tasks may include: capitalization and punctuation, which improves readability; voice activity detection (VAD) and end-of-query (EOQ) detection, which are important for implementing low-latency systems; and natural conversation understanding, which involves predicting the cadence and turn-taking aspects of an ongoing conversation. In this study, we focus on improving the quality of such auxiliary tasks in an end-to-end (E2E) ASR setting via text injection.

We build on two recent capabilities for speech models. First is the E2E integration of auxiliary tasks with the ASR task into a single model. In the past, auxiliary tasks were usually performed by separate models downstream of ASR [2, 3, 4, 5]. Recent work has successfully explored integrating auxiliary tasks, such as endpointing [6, 7, 8], capitalization [9], natural conversation understanding [10], and speaker diarization [11] into the same model as ASR prediction. E2E integration of ASR and auxiliary tasks has a key drawback, however. When folded into an E2E ASR model, pure text-to-text tasks (such as capitalization) can no longer be trained on plentiful text-only data (i.e., text data with no associated audio); instead, their training examples will be limited to the transcripts available in paired audio-text labeled data. This puts E2E methods at a disadvantage, since text-only data is generally more plentiful and easier to obtain than labeled audio data, and can be used to more easily expose the model to rare words and other long-tail phenomena which may be difficult to collect in labeled audio form [12].

The second capability enabling the current study is the use of "text injection" as a means of improving ASR quality [13]. An ASR model's internal language model (ILM) is the notional part of the network that predicts the next token given the previous token history, independent of audio input. Though it is usually infeasible to exactly separate the influence of audio input from previous token predictions, several methods have been developed to estimate ILM scores [14, 15]. Text-only data can then be used to refine the ILM capabilities of the ASR network [16, 17].

In this work, we propose a method to utilize text injection techniques for improving auxiliary task performance in an E2E ASR model. Doing so allows auxiliary tasks to access the multi-task learning benefits of co-training with ASR while still including rich text-only data in their training corpora. We focus our study on two tasks: capitalization and conversational turn-taking prediction. The former is a strongly text-based task, since capitalization is merely a form of de-normalization from spoken to written domain, and capitalized words are not pronounced differently. The latter task clearly involves combining linguistic and acoustic understanding — the prosody of the input speech as well as the semantics of the current recognition are both informative for predicting whether a pause is only momentary or if the user has finished speaking. We integrate these tasks into a production-ready model, streaming E2E RNN-T ASR model [18, 19]. We show results demonstrating that text injection can meaningfully improve auxiliary task performance, particularly in long-tail settings.

2. Related Work

While auxiliary tasks are usually performed by separate models from ASR [20, 21], E2E approaches to auxiliary task modeling have been recently popular for production-grade systems. Joint training of ASR with endpointing [7], capitalization [9, 22], intended query detection [23, 24], sentence segmentation [25], and more, have been explored. Our work builds most closely on Wang et al.[9], who co-train ASR, capitalization, and turn-taking prediction by building multiple parallel label sequences. To our knowledge, this is the first attempt to refine auxiliary tasks in an E2E ASR model using text-only data.

There has long been interest in utilizing unpaired text data for the ASR task. Several approaches to LM fusion, the use of an external LM to improve ASR recognition quality, have been proposed [26]. These methods have the drawback of increasing total parameter count (due to the size of the LM model), and computation cost during inference. Text injection [13] solves these issues by using LM-style unpaired text data to train the

ASR model itself. Some methods focus on fine-tuning an existing ASR model trained on audio-text data; ILM adaptation of the ASR decoder has been shown to work well [27, 28, 29]. The text injection method we employ here is joint end-to-end and ILM training (JEIT), which was introduced by Meng et al [30]. We choose JEIT as our method due to its lightweight nature; its primary focus on refining the ASR decoder makes comparison to standard methods straightforward, since the behavior of the audio encoder is preserved. Other methods inject text data directly into the encoder, with fixed and learned duration models to align text and audio sequences [16, 17]. All of the above works focus on improving ASR quality, both for standard and long-tail data; to the best of our knowledge, adapting these techniques for auxiliary tasks is a novel contribution to the literature.

3. Auxiliary Tasks

3.1. Capitalization

Capitalization is the process of restoring the correct case (uppercase or lowercase) of noisy text. Notably, capitalization is specific to the written domain, and has no marker in spoken speech. This task is important for maintaining readability in ASR output, especially for long-form captioning cases.

3.2. Conversational turn-taking

Turn-taking is an active area of research for E2E speech modeling [10, 31]. While humans typically adjust their speech when interacting with voice assistants [31], natural human speech patterns during conversation are often filled with natural disfluencies. For digital assistant products, it is desirable that voice assistants have the ability to predict when the speaker is expecting a response, versus when they merely pause with the intention to resume speaking. We model this phenomenon similar to Chang et al. [10], who classify pauses in speech as being within a complete thought, or after having a finished complete thought. That is, when a user stops speaking, the model should predict whether they will continue speaking after a brief pause or whether a system response is expected. Because the active region of interest is pauses in the audio, we refer to this task in this paper as "pause prediction."

4. Model

4.1. Multi-output HAT decoder

HAT is a decoder structure for RNN-T in which the $\langle\texttt{blank}\rangle$ probability is computed separately from next token prediction, facilitating more accurate ILM estimation [14]. Wang et al. [9] propose a variant of HAT decoder which introduces multiple joint networks, one for each task (in our case, these are ASR, capitalization, and pause prediction). All of the parallel joint networks are conditioned on features from both the prediction network and audio encoders.

The model is trained using an RNN-T objective [18], where at each timestep the model may choose to emit a wordpiece token, or to insert a special token $\langle\texttt{blank}\rangle$ which indicates non-emission. Formally, let X be the input utterance and Y be the label sequence. The ASR output space $\mathcal{Y}_{\text{ASR}}$ consists of $\{y^0 = \langle\texttt{blank}\rangle, y^1, y^2, ...\}$.. Let $T = |X|$ be the number of input audio frames and $U = |Y|$ be the length of the transcript. The acoustic encoder produces $f(X) = [f_0, ..., f_{T-1}]$, $f_t \in \mathcal{R}^{D_a}$, and the prediction network produces $g(X) = [g_0, ..., g_{U-1}]$, $g_u \in \mathcal{R}^{D_p}$. As in the original HAT implementation, the joint

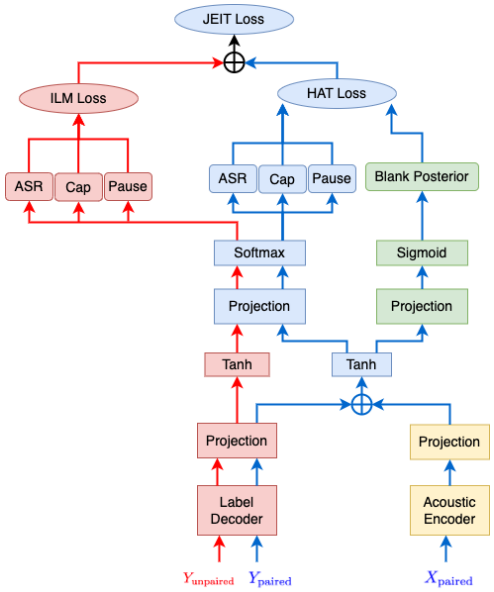

Figure 1: *Model diagram for JEIT training. The blue arrows denote the data flow for paired audio-text data. The red arrows denote the path that unpaired text data takes through the network. Baseline experiments are trained using only the blue paths, while the proposed system is trained using both.*

network fuses f_t and g_u with a "project and sum" operation to produce a hidden representation $h_{t,u}$, which is then passed through a non-linear activation and a final linear layer to produce $s_{t,u}$:

$$h_{t,u} = P \cdot f_t + Q \cdot g_u + b_h \quad \in \mathcal{R}^{D_h} \tag{1}$$

$$s_{t,u} = A \cdot \tanh(h_{t,u}) + b_s \quad \in \mathcal{R}^{V}. \tag{2}$$

where P, Q, and A are learned weight matrices with dimensions determined by D_a, D_p, D_h, and V is the size of the vocabulary. As this is a HAT model, the 0-th logit of $s_{t,u}$ is used individually to compute the probability of emission $b_{t,u}$:

$$b_{t,u} := P_{t,u}(\langle\texttt{blank}\rangle | f_{0:t}, g_{0:u}) = \sigma(s_{t,u}[0]) \tag{3}$$

where $\sigma(x) = 1/(1 + \exp(-x))$ is the sigmoid activation. Probabilities over the ASR tokens are computed by feeding all remaining logits to a softmax function. The probability of each ASR token y_v in the vocabulary is:

$$\begin{aligned}\hat{y}_{v;t,u} &= P_{t,u}(\hat{y}_v | f_{0:t}, g_{0:u}) \\ &= \text{softmax}(s_{t,u}[1:])[v-1]\end{aligned} \tag{4}$$

Thus the predicted probability distribution over all output tokens is the emission probability, followed by the probabilities of each token given emission:

$$\begin{aligned}\hat{y}_{t,u} = [b_{t,u}, \quad & (1 - b_{t,u}) \cdot \hat{y}_{0;t,u}, \\ \dots, \quad & (1 - b_{t,u}) \cdot \hat{y}_{V-1;t,u}]\end{aligned} \tag{5}$$

Thus far we have referred to the mechanism above in terms of ASR prediction. Capitalization and pause predictions are made in the exact same way, where each task independently computes Eqs. (1) and (2) based on the shared representations f_t and g_u (note that each auxiliary task is exposed to the label history of the ASR output, not its own prediction history).

Since capitalization tokens must be strictly aligned with ASR tokens, the capitalization posterior borrows the blank logit

Initial Transcript	driving time to san francisco					
Capitalization	Driving time to San Francisco					
Tokenization	⟨cap⟩ _driving _time _to ⟨pause⟩ ⟨cap⟩ _san ⟨cap⟩ _fran cisco ⟨eos⟩					
Label Factorization	Y_{ASR} :	_driving	_time	_to	_san	_fran	cisco
	Y_{Cap} :	⟨cap⟩	⟨non-cap⟩	⟨non-cap⟩	⟨cap⟩	⟨cap⟩	⟨non-cap⟩
	Y_{Pause} :	⟨non-pause⟩	⟨non-pause⟩	⟨pause⟩	⟨non-pause⟩	⟨non-pause⟩	⟨eos⟩

Figure 2: *Data preparation for auxiliary tasks. Wordpieces that begin with _ denote word boundaries. In this example, we assume that the speaker takes a verbal pause as follows: "Driving time to... San Francisco," to illustrate the ⟨pause⟩ logic.*

from the ASR prediction. Thus, a capitalization token will only be emitted when an ASR token is emitted as well. Capitalization has output space $\mathcal{Y}_{\text{Cap}} = \{\langle\texttt{cap}\rangle, \langle\texttt{non-cap}\rangle\}$ and its posterior is:

$$\hat{y}^{\text{Cap}}_{t,u} = [b^{\text{ASR}}_{t,u}, \quad (1 - b^{\text{ASR}}_{t,u}) \cdot P_{t,u}(\langle\texttt{cap}\rangle), \\ (1 - b^{\text{ASR}}_{t,u}) \cdot P_{t,u}(\langle\texttt{non-cap}\rangle)] \tag{6}$$

At inference time, we estimate $P(\langle\texttt{cap}\rangle)$ every time an ASR token is emitted and predict a capitalization if it is above a threshold (in this work, we use 0.5).

Pause tokens do not need to be strictly aligned with the ASR transcript prediction, since they are likely to be predicted during non-speech periods in the audio during inference, so the turn-taking sequence has its own blank posterior. The pause prediction output space is $\mathcal{Y}_{\text{Pause}} = \{\langle\texttt{blank}\rangle, \langle\texttt{non-pause}\rangle, \langle\texttt{pause}\rangle, \langle\texttt{eos}\rangle\}$ and its posterior is computed in the same way as Eq. (5).

5. Training

5.1. JEIT

Joint end-to-end model and ILM training (JEIT) was proposed by Meng et al. [30] as a way to train an RNN-T ASR model on paired audio-text data while simultaneously training the HAT decoder ILM on text-only data. For paired dataset $\mathcal{D}_{\text{paired}}$, training is conducted in the usual way; the model is given the audio sequence as input and predicts $P_{\text{E2E}}(Y|X)$. This is converted to a loss $\mathcal{L}^{\text{ASR}}_{\text{E2E}}$ via the RNN-T objective [18].

The text-only dataset $\mathcal{D}_{\text{unpaired}}$ contains transcripts with capitalization and pause annotations (see §5.2). Similar to HAT ILM adaptation (ILMA) [27], we feed the transcript as the previous token history to the prediction network, and mock the encoder output with vectors full of zeros: $\forall_{t \in 0:T} : f_t = \mathbf{0}$. Since the audio sequence does not exist, we simply ignore the blank posterior, and the predicted next token probabilities are given directly by the softmax output in Eq. (4). With previous token history as input and next token probabilities as output, this allows us to estimate $P_{\text{ILM}}(y_t : y_{0:t-1})$. ILM loss is defined as the negative log probability of each label token given the label sequence prefix:

$$\mathcal{L}^{\text{ASR}}_{\text{ILM}} = -\sum_{u=1}^{U} \log P(y^{\text{ASR}}_u | \hat{y}^{\text{ASR}}_{0:u-1}) \tag{7}$$

The losses $\mathcal{L}_{\text{E2E}}$ and $\mathcal{L}_{\text{ILM}}$ are averaged over their respective datasets $\mathcal{D}_{\text{paired}}$ and $\mathcal{D}_{\text{unpaired}}$, then combined in a weighted average to obtain the total JEIT loss:

$$\mathcal{L}^{\text{ASR}}_{\text{JEIT}}(\mathcal{D}_{\text{paired}}, \mathcal{D}_{\text{unpaired}}) = \\ \mathcal{L}^{\text{ASR}}_{\text{E2E}}(\mathcal{D}_{\text{paired}}) + \beta \mathcal{L}^{\text{ASR}}_{\text{ILM}}(\mathcal{D}_{\text{unpaired}}) \tag{8}$$

where β is a hyperparameter controlling the weight given to ILM training (in this work, we use $\beta = 0.2$ to match Meng et al.'s original study).

Adapting JEIT to include auxiliary tasks is straightforward. As described in §4.1, each auxiliary task makes a sequence prediction Y_{Aux} based on the predicted ASR sequence Y_{ASR}. Thus, each auxiliary task predicts $P_{\text{E2E}}(Y_{\text{Aux}}|\hat{Y}_{\text{ASR}}; X)$ to produce $\mathcal{L}^{\text{Aux}}_{\text{E2E}}$. Similarly, the ILM loss is

$$\mathcal{L}^{\text{Aux}}_{\text{ILM}} = -\sum_{u=1}^{U} \log P(y^{\text{Aux}}_u | \hat{y}^{\text{ASR}}_{0:u-1}) \tag{9}$$

The full JEIT loss for each task is defined in the same way as Eq. (8). Total loss is a linear combination of all tasks: (datasets omitted for clarity):

$$\mathcal{L}^{\text{Total}}_{\text{JEIT}} = \mathcal{L}^{\text{ASR}}_{\text{E2E}} + \beta \mathcal{L}^{\text{ASR}}_{\text{ILM}} + \alpha_{\text{Cap}}(\mathcal{L}^{\text{Cap}}_{\text{E2E}} + \beta \mathcal{L}^{\text{Cap}}_{\text{ILM}}) \\ + \alpha_{\text{Pause}}(\mathcal{L}^{\text{Pause}}_{\text{E2E}} + \beta \mathcal{L}^{\text{Pause}}_{\text{ILM}}) \tag{10}$$

where each α is a loss weight for the corresponding task. Matching Wang's original study, we use $\alpha_{\text{Cap}} = 0.1$ and $\alpha_{\text{Pause}} = 0.3$. Figure 1 shows the data flow for paired and unpaired data through the ASR model.

5.2. Transcript annotation

While a small amount of our paired training corpus is hand-labeled and capitalized, most of our paired data and all of our unpaired text data have lowercase transcripts. For the lowercase transcripts, we use a text-based truecasing RNN teacher model similar to [32] to produce capitalization predictions.

Producing pause prediction labels requires different approaches for paired and unpaired data. For paired audio-text data, we use the approach taken by Chang et al. [10], which uses heuristics based on a forced alignment [33] to insert pause tokens into the transcript. There are two such tokens: ⟨pause⟩ denotes a brief stop by the speaker in the middle of a full thought, and ⟨eos⟩ (end of sentence) is inserted at the end of the full thought, i.e. a full conversational turn.

For unpaired text-only data, the above strategy is impossible, since we do not have access to the associated audio. Instead, we rely on the fact that our text-only data comes from short-query sources (see §6.2). We simply append the ⟨eos⟩ token to the end of the transcript.

5.3. Multi-task label structure

A common approach to transcript labeling for auxiliary tasks would be to embed special tokens corresponding to each task in the transcript itself [7]. However, this is not ideal for inference, since the extra tokens must be expanded in-line with the ASR tokens; if predictions on competing beams differ only in their special tokens, lattice diversity is reduced because the ASR prediction would be identical. To solve for this, we follow Wang et al. [9], factorizing the auxiliary task tokens into parallel sequences of equal length, one for each task. The ASR task is trained on the lowercase transcript sequence, segmented into wordpieces. The capitalization sequence is defined as follows: each token is either ⟨cap⟩ (capitalized) or ⟨non-cap⟩

Table 1: *Capitalization. We report word error rate (WER (%)) and uppercase error rate (UER (%)) on a representative ("head") voice dictation dataset. We also report UER on a dataset containing rare words ("tail").*

Exp.	Method	WER	Head UER	Tail UER
B1	Paired Data Only	3.9	24.3	46.0
E1	JEIT (Proposed)	3.9	24.7	45.1

Table 2: *Sample capitalization improvements. For anonymity, some transcript words are substituted with equivalents, while preserving the capitalization dynamics.*

Ground Truth	Hypothesis	Exp.
Matheus Nicolau UFC fighter	Matheus nicolau UFC fighter	B1
	Matheus Nicolau UFC fighter	E1
Smoketown Brewing Company	smoketown Brewing Company	B1
	Smoketown Brewing Company	E1
Play Maldita Vecindad	play Maldita vecindad	B1
	play Maldita Vecindad	E1

(not capitalized), based on the corresponding wordpiece in the ASR transcript. Similarly, the turn-prediction sequence is populated with $\langle$pause$\rangle$ and $\langle$eos$\rangle$ tokens corresponding to the wordpieces immediately preceding the corresponding predicted pauses in the transcript. All other token slots are filled with $\langle$non-pause$\rangle$. The successive steps of label generation are shown in Figure 2.

6. Experimental Details

6.1. Model architecture

We use a 128-dimensional log-mel feature frontend computed on 32ms windows with a 10ms stride. We stack four consecutive frames together and sub-sambled by a factor of 3, resulting in 512-dim features at a 30ms framerate. This vector is then concatenated with a 16-dim one-hot domain ID vector [34]. As our ASR backbone we use a 2-pass cascaded encoder model [35]. The first encoder consists of 7 conformer layers [36] with causal convolution and left-context attention. The second encoder consists of 10 conformer layers with a 900ms lookahead. Each conformer layer uses 512-dim 8-head self-attention and a kernel size of 15, and the final layer emits $D_a = 384$-dim encodings. The prediction network of each decoder is a V^2 embedding lookup table, which computes $D_p = 640$-dim features based on embeddings of the previous two wordpiece tokens. Each joint network has hidden dimension $D_h = 384$, and predictions are made over a vocabulary of $V = 4096$ wordpieces. For evaluation, we report only 2nd pass WER.

In total, our model has $\sim$160M params. It is implemented in Tensorflow using the Lingvo toolkit, and is trained on proprietary specialized hardware for 500k steps using batch size 4096 for paired and unpaired data.

6.2. Data

6.2.1. Paired training data

Our training set of audio-text pairs consists of a dataset of 650 million English multi-domain examples, drawn from search, dictation, online video, and telephony domains. A small subset of these utterances are anonymized and hand-transcribed, and the rest are pseudo-labeled by a 600M parameter bidirectional teacher model. To increase model robustness, we apply simulated noise to utterances, as well as SpecAug [37].

Table 3: *Pause prediction. We report precision and recall for the $\langle$eos$\rangle$ token on a conversation-style test set.*

Exp.	Method	$\langle$eos$\rangle$ Prediction	
		Precision	Recall
B1	Paired Data Only	72.16	89.56
E1	JEIT (Proposed)	71.12	92.94

6.2.2. Unpaired training data

Our text-only data selection pipeline is designed in the style of Sentence-Select by Huang et al [12]. Text query data ($\sim$ 100B utterances) is collected from web search, maps search, app store search, and online video search domains. This data is filtered for rare words and contrastive filtering based on perplexity is applied. Because the data is selected to include rare words, we expect improvements at the tails of the evaluation distribution.

6.2.3. Evaluation Data

WER is reported on $\sim$ 17k utterances representative of real-world voice dictation traffic. Ground truth transcript and auxiliary task annotations are obtained via human labeling. We also report uppercase error rate (UER) on this set, which is calculated by removing all lowercase letters from the ground truth label and the predicted transcript and computing standard WER with upper case letters as words. Since our text-only data focuses on long-tail traffic, we also report UER on a set of $\sim$300 utterances with transcripts containing rare words.

For pause prediction, we use a testset of $\sim$2500 utterances containing hesitations and multiple consecutive commands. Pauses in the audio are hand-annotated as continuation pauses or final pauses. The metrics reported are average precision and recall of the $\langle$eos$\rangle$ token.

7. Results

We evaluate the proposed method (E1) against a baseline (B1) which uses an identical model but is trained on paired data only (Table 1). On the large voice search test set on which it is evaluated, WER does not change, while UER regresses slightly on the voice dictation dataset (1.6% relative). For long tail data, UER improves by a relative 2.0%. Table 2 shows example transcripts demonstrating our proposed method's better capability at recognizing capitalized named entities.

Pause detection recall improves by 3.7% (relative), while precision is reduced slightly, by 1.4% (relative) (Table 3). This matches the intuition that our text-injection method biases the model towards $\langle$eos$\rangle$, since the unpaired text data is only augmented with $\langle$eos$\rangle$ at the end of short form transcripts. However, the improvement in recall is larger than the change in precision, and in a production setting, hyperparameters may be tuned to balance the two metrics differently. These results show that augmenting the training data of an ASR model with unpaired text data using JEIT can be used to meaningfully improve pause prediction performance, without regressing word-error rate.

These results show that augmenting the training data of an ASR model with unpaired text data meaningfully impacts auxiliary task performance. In our case, we use long-tail, shortform text data to improve capitalization performance for rare words and turn-taking prediction recall. We recommend that future work extend this technique to other text injection methods, and explore the use of text injection for other auxiliary tasks.

8. References

[1] J. Schalkwyk, D. Beeferman, F. Beaufays, B. Byrne, C. Chelba, M. Cohen, M. Kamvar, and B. Strope, *"Your Word is my Command": Google Search by Voice: A Case Study*. Boston, MA: Springer US, 2010, pp. 61–90.

[2] S.-Y. Chang, B. Li, T. N. Sainath, G. Simko, and C. Parada, "End-point detection using grid long short-term memory networks for streaming speech recognition." in *Proc. Interspeech*, 2017.

[3] F. Beaufays and B. Strope, "Language model capitalization," in *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, 2013, pp. 6749–6752.

[4] B. P. Nguyen, V. B. H. Nguyen, H. Nguyen, P. N. Phuong, T.-L. Nguyen, Q. T. Do, and L. C. Mai, "Fast and accurate capitalization and punctuation for automatic speech recognition using transformer and chunk merging," *2019 22nd Conference of the Oriental COCOSDA International Committee for the Co-ordination and Standardisation of Speech Databases and Assessment Techniques (O-COCOSDA)*, pp. 1–5, 2019.

[5] F. Batista, "Recovering capitalization and punctuation marks on speech transcriptions," 2011.

[6] S. Bijwadia, S.-y. Chang, B. Li, T. Sainath, C. Zhang, and Y. He, "Unified end-to-end speech recognition and endpointing for fast and efficient speech systems," in *2022 IEEE Spoken Language Technology Workshop (SLT)*, 2023, pp. 310–316.

[7] S. Chang, R. Prabhavalkar, Y. He *et al.*, "Joint Endpointing and Decoding with End-to-End Models," in *Proc. ICASSP*, 2019.

[8] R. Maas, A. Rastrow, C. Ma, G. Lan, K. Goehner, G. Tiwari, S. Joseph, and B. Hoffmeister, "Combining acoustic embeddings and decoding features for end-of-utterance detection in real-time far-field speech recognition systems," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 5544–5548.

[9] W. Wang, D. Zhao, S. Ding, H. Zhang, S. yiin Chang, D. Rybach, T. N. Sainath, Y. He, I. McGraw, and S. Kumar, "Multi-output rnn-t joint networks for multi-task learning of asr and auxiliary tasks," in *Proc. ICASSP (forthcoming*, 2023.

[10] S. yiin Chang, B. Li, T. N. Sainath, C. Zhang, T. Strohman, Q. Liang, and Y. He, "Turn-taking prediction for natural conversational speech," in *Interspeech*, 2022.

[11] L. E. Shafey, H. Soltau, and I. Shafran, "Joint speech recognition and speaker diarization via sequence transduction," *ArXiv*, vol. abs/1907.05337, 2019.

[12] W. R. Huang, C. Peyser, T. N. Sainath, R. Pang, T. Strohman, and S. Kumar, "Sentence-select: Large-scale language model data selection for rare-word speech recognition," in *Interspeech*, 2022.

[13] S. Thomas, B. Kingsbury, G. Saon, and H.-K. J. Kuo, "Integrating text inputs for training and adapting rnn transducer asr models," *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 8127–8131, 2022.

[14] E. Variani, D. Rybach, C. Allauzen, and M. Riley, "Hybrid Autoregressive Transducer (HAT)," in *Proc. ICASSP*, 2020.

[15] Z. Meng, S. Parthasarathy, E. Sun, Y. Gaur, N. Kanda, L. Lu, X. Chen, R. Zhao, J. Li, and Y. Gong, "Internal language model estimation for domain-adaptive end-to-end speech recognition," *2021 IEEE Spoken Language Technology Workshop (SLT)*, pp. 243–250, 2020.

[16] T. N. Sainath, R. Prabhavalkar, A. Bapna, Y. Zhang, Z. Huo, Z. Chen, B. Li, W. Wang, and T. Strohman, "Joist: A joint speech and text streaming model for asr," *2022 IEEE Spoken Language Technology Workshop (SLT)*, pp. 52–59, 2022.

[17] Z. Chen, Y. Zhang, A. Rosenberg, B. Ramabhadran, P. Moreno, A. Bapna, and H. Zen, "Maestro: Matched speech text representations through modality matching," in *ICASSP*, 2022.

[18] A. Graves, "Sequence Transduction with Recurrent Neural Networks," *CoRR*, vol. abs/1211.3711, 2012.

[19] T. N. Sainath, Y. He, B. Li, A. Narayanan, R. Pang, A. Bruguier, S.-y. Chang, W. Li, R. Alvarez, Z. Chen *et al.*, "A streaming on-device end-to-end model surpassing server-side conventional model quality and latency," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 6059–6063.

[20] R. Rei, N. M. Guerreiro, and F. Batista, "Automatic truecasing of video subtitles using bert: A multilingual adaptable approach," *Information Processing and Management of Uncertainty in Knowledge-Based Systems*, vol. 1237, pp. 708 – 721, 2020.

[21] M. Sunkara, S. Ronanki, K. Dixit, S. Bodapati, and K. Kirchhoff, "Robust prediction of punctuation and truecasing for medical asr," *ArXiv*, vol. abs/2007.02025, 2020.

[22] V. Pahuja, A. Laha, S. Mirkin, V. C. Raykar, L. Kotlerman, and G. Lev, "Joint learning of correlated sequence labeling tasks using bidirectional recurrent neural networks," in *Interspeech*, 2017.

[23] S. yiin Chang, G. Prakash, Z. Wu, Q. Liang, T. N. Sainath, B. Li, A. Stambler, S. Upadhyay, M. Faruqui, and T. Strohman, "Streaming intended query detection using e2e modeling for continued conversation," in *Interspeech*, 2022.

[24] S. H. R. Mallidi, R. Maas, K. Goehner, A. Rastrow, S. Matsoukas, and B. Hoffmeister, "Device-directed utterance detection," in *Interspeech*, 2018.

[25] W. R. Huang, S. yiin Chang, D. Rybach, R. Prabhavalkar, T. N. Sainath, C. Allauzen, C. Peyser, and Z. Lu, "E2e segmenter: Joint segmenting and decoding for long-form asr," in *Interspeech*, 2022.

[26] A. Kannan, Y. Wu, P. Nguyen *et al.*, "An analysis of incorporating an external language model into a sequence-to-sequence model," in *Proc. ICASSP*, 2018.

[27] Z. Meng, Y. Gaur, N. Kanda, J. Li, X. Chen, Y. Wu, and Y. Gong, "Internal language model adaptation with text-only data for end-to-end speech recognition," *ArXiv*, vol. abs/2110.05354, 2021.

[28] V. Bataev, R. Korostik, E. Shabalin, V. Lavrukhin, and B. Ginsburg, "Text-only domain adaptation for end-to-end asr using integrated text-to-mel-spectrogram generator," *ArXiv*, vol. abs/2302.14036, 2023.

[29] Z. Meng, T. Chen, R. Prabhavalkar, Y. Zhang, G. Wang, K. Audhkhasi, J. Emond, T. Strohman, B. Ramabhadran, W. R. Huang, E. Variani, Y. Huang, and P. J. Moreno, "Modular hybrid autoregressive transducer," 2022. [Online]. Available: https://arxiv.org/abs/2210.17049

[30] Z. Meng, W. Wang, R. Prabhavalkar, T. N. Sainath, T. Chen, E. Variani, Y. Zhang, B. Li, A. Rosenberg, and B. Ramabhadran, "Jeit: Joint end-to-end model and internal language model training for speech recognition," 2023. [Online]. Available: https://arxiv.org/abs/2302.08583

[31] C. Liu, C. T. Ishi, and H. Ishiguro, "Turn-taking estimation model based on joint embedding of lexical and prosodic contents," in *Interspeech*, 2017.

[32] H. Zhang, Y.-C. Cheng, S. Kumar, W. R. Huang, M. Chen, and R. Mathews, "Capitalization normalization for language modeling with an accurate and efficient hierarchical rnn model," *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 6097–6101, 2022.

[33] P. J. Moreno, C. F. Joerg, J.-M. Van Thong, and O. Glickman, "A recursive algorithm for the forced alignment of very long audio segments." in *ICSLP*, vol. 98, 1998, pp. 2711–2714.

[34] A. Narayanan, R. Prabhavalkar, C.-C. Chiu *et al.*, "Recognizing Long-Form Speech Using Streaming End-to-End Models," in *Proc. ASRU*, 2019.

[35] A. Narayanan, T. N. Sainath, R. Pang *et al.*, "Cascaded encoders for unifying streaming and non-streaming ASR," in *Proc. ICASSP*, 2021.

[36] A. Gulati, J. Qin, C.-C. Chiu *et al.*, "Conformer: Convolution-augmented Transformer for Speech Recognition," in *Proc. Interspeech*, 2020.

[37] D. S. Park, W. Chan, Y. Zhang *et al.*, "SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition," in *Proc. Interspeech*, 2019.